VIVA

Complex devices

PUBLICATION ISSUED BY:

Via Kennedy, 24 - 10019 STRAMBINO (TO)
Tel.: +39 0125 63 68 11
Fax.: +39 0125 63 68 99
e-mail: seica@seica.com
Web site: http://www.seica.com

Copyright © 2007
All rights reserved

Microsoft Windows is a registered trademark of Microsoft Corporation

Document code: MA-VI-VIVADCEN-00
Release software: VIVA Rel. 1.4.0 for Windows 2000/XP'
Date of issue: 13-12-2007

Seica S.p.A. reserves the right to change the technical specifications without notice.

CONTENTS

CONTENTS I
LIST OF FIGURES I
LIST OF TABLES I
1. Complex devices in VIVA Environment 1-1
1.1 VIVA – reference database, components and complex devices 1-2
1.2 VIVA - Graphical User Interface 1-9
2. Functions and complex devices 2-1
2.1 Simple devices and complex devices 2-2
2.2 Complex device made up by discrete components 2-3
2.2.1 Add functions to board data base 2-4
2.2.2 Add a complex device to the board data base 2-5
2.2.3 Assign a device to a complex device 2-7
2.2.4 Assign a component to a complex device 2-8
2.2.4.1 Assign a function to a component 2-9
2.2.4.2 Assign a macro to a component 2-10
2.3 Complex device made up by ic 2-11
2.3.1 Add functions to board database 2-15
2.3.2 Add a complex device to board database 2-15
2.3.3 Assign components to complex devices 2-18
2.3.4 Complex device made up by several macros 2-20
3. Bibliography 3-1

LIST OF FIGURES[1]

Fig. 1-1 - VIVA: Program Preparation, Verification and Execution 1-5
Fig. 1-2 - VIVA: Complex Components, New Device and Complex Device 1-8
Fig. 1-3 - VIVA: Graphical User Interface 1-9
Fig. 2-1 - VIVA: Devices and Components list for two components of a complex device 2-3
Fig. 2-2 - VIVA: Complex device and VIVA models for Operational Amplifier 2-12
Fig. 2-3 - VIVA: Device and Component list after “import Data” 2-13
Fig. 2-4 - VIVA: “OpAmp” and “Digital” function from “Funcion & Macro data base” 2-13
Fig. 2-5 - VIVA: “Assign Function” tables 2-14
Fig. 2-6 - VIVA: “Debug Environment” 2-14
Fig. 2-7 - VIVA: Functions in Programming Environment 2-15
Fig. 2-8 - VIVA: Complex device “Operational Amplifier” used like Amplifier 2-16
Fig. 2-9 - VIVA: Complex device “Operational Amplifier” used as Comparator 2-16
Fig. 2-10 - VIVA: Complex device “Operational Amplifier” used as Voltage Follower 2-17
Fig. 2-11 - VIVA: “Assign Function” Component table 2-18
Fig. 2-12 - VIVA: Device and Component list after “Assign Function” 2-18
Fig. 2-13 - VIVA: “Debug Environment” 2-19

LIST OF TABLES

Tab. 1-1 - VIVA: Seica CAD data file (*.nod) 1-6
Tab. 1-2 - VIVA: Seica CAD data file (*.par) 1-6

[1] The figures included in this manual may not correspond to the last product release. Their aim is to support the topics developed in the manual.

1. Complex devices in VIVA Environment

The Programming Environment available in the VIVA Software, used to manage the test systems designed and manufactured by Seica S.p.A., allows to generate test programs to check UUTs made up by components grouped into devices.
For each device there is an associated function used to check the components which belong to a specific device.
The functions are formed by one or more macros which make up the code executed to check the components.
It is possible to set the macros according to the features of the components to be checked (scale to be measured, accepted tolerances for the measured value, component pins identification, etc…)

The devices included in the Programming Environment database form a set of models that in many cases do not exactly correspond to the actual components on the UUT.

In general, the components installed on a UUT may differ from the theoric models for several reasons:

- The numbering of actual pins is different from that of the reference model included into the Programming Environment.
- The actual component contains more instances than the reference model.
- The actual component is made up by several sub-components associated to different reference models available in the Programming Environment
- Etc…

To execute the functions required to check the components on a UUT, the “Complex Devices” are available for the user. They allow to solve the problems of mapping between the actual components and the models available in the “Programming Environment”.

Before describing the complex devices, we provide some useful information (see Bibliography [1], to better understand the complex devices.

1.1 VIVA – reference database, components and complex devices

The flowchart reproduced in Fig. 1-1 schemes the activities which can be executed with VIVA SW to develop and use a test program. On the right side of the flow chart, are highlighted the groups of data imported to define: the components to be checked, the functions used for checking, the system macros used by functions and the values assigned to macro settings.

In particular:

External CAD data files

These are the CAD files of the board to be tested and contain the information required to locate the board nets, and the values to be checked. As an example you can find the description of the fields used in the Seica CAD[2] format, which schedules two kinds of records:

- One, to physically identify the board nodes (file.nod).
- Another to identify the components which make up the board (file.par).

The following fields of the record contain information about the nodes of a board:

SIGNAL	Identifies the name of the signal on the node; it is a reference to the board electric schemes.
COMPONENT	Identifies the name of the component associated to the node. The couple COMPONENT/PIN univocally identifies the name of the node.
PIN	Identifies the pin number of the component associated to the node. The couple COMPONENT/PIN univocally identifies the name of the node.
X	Identifies, in the space X,Y containing the board, the x-axis of the node. The couple “X”- “Y” identifies the node position.
Y	Identifies, in the space X,Y containing the board, the y-axis of the node. The couple “X”- “Y” identifies the node position.
PAD	Defines the size and shape of the PAD associated to the node.
POS	Identifies on which side of the board is located the node (T for Top, B for Bottom O for bOth).
TECN	Identifies the technology of the component mounted on the node (T for Trough hole and S for SMD).
TEST	Identifies the elaboration required on the node (Y for Test and N for Don’t test).
CHANNEL	node identification number (also called Channel) defined by the system. This field is not relevant in the external CAD files and is filled in to import data into the Board CAD data base

It is necessary to fill in all the fields (10)!

[2] The CAD Seica format has been taken as a reference since, after the “Import Data” phase, both from generic CAD format or manual input, the CAD Seica format is created to describe the UUT.

The following fields of the record contain information about the components of a board:

COMPONENT	identifies the name of the component associated to the node. It is the same name you find in the field "Component" of the record which identifies the node on the board.
DEVICE[3]	Identifies a typology of components to which the component belongs. In the VIVA environment a function is associated to each "DEVICE" to test the components belonging to "DEVICE".
VALUE	Identifies the value of the component setting to be checked.
UNIT	Identifies the scale value of the setting to be checked.
TOL+	Identifies the positive tolerance of the value to be checked.
TOL-	Identifies the negative tolerance of the value to be checked.
NRPIN	Identifies the number of component pins.
MACRO/FUNCTION	Identifies either the macro or the function associated to the component. If this field is not filled in the function associated to the component is that defined in "Function & Macro data base"

The first seven fields must be filled in! Only the field "MACRO/FUNCTION" may be left blank.

Notes on field MACRO/FUNCTION:

(This field may contain the name of the macro or of the function associated to the component)

Name	The **macro** "Name" is used to check the component of the record containing "Name".
$Name	The **function** "Name" is used to check the component of the record containing "Name". The dialog "Assign Function" allows to check that the function is associated to the component and that the device has a new name: "**<old name device>_<Component name>**". (e.g. XDIODERESISTORS_ACDC2).

If the field "MACRO/Function" contains a value, this value has priority on the associations defined with "Patterns" in the "Programming Environment". If the field "MACRO/Function" contains a name not included in the Programming Environment database, after the "Import Data" phase the dialog "Assign Function" allows to associate to the component:

- The required function
- The required macro
- The required complex device

When the field "MACRO/FUNCTION" defines a function for a component, with the function "Export Data", in the exported file .par:

1. The field "MACRO/FUNCTION" is not filled in
2. the value of the belonging device will have the new name. (so it is lost)

Board CAD data base

These information are related to the board to be tested; they are used to locate the nodes on the board and the values to be checked. These is the same kind of information included in the "Seica CAD data files" and in "External CAD data files" but are stored in the board data base. It is possible to insert the information either through the "Import CAD data" function or the "Import Manual Data" function. The "Edit Board" environment allows to modify the information stored in the "Board CAD data base" but does not modify the information included in the "Seica CAD data files" (not modified by VIVA SW after the "Import Data" phase).

[3] In the "Board CAD data base" it is possible to find "Complex Devices", which identify devices made up by one or more parts (SUB*i*). To each part (SUB*i*) are associated the pins of the device which make up the part and the function used to check the part. These "Complex Devices" are created and included only in "Board CAD data base" and not in the "External CAD data files".

Function & Macro data base

These information are related to the definition of functions, the components associated to the functions, and the value of the macro settings. The information included in this group of data, together with those included in the group “Board CAD data base” are used to create the test program identified by the group of data “Program data base”.

In “Function & Macro data base” the user may insert the complex devices used when creating the test program.

In the phase “CREATE A NEW BOARD”, the Function & Macro data base is initialized by copying the model identified by the “Template” selected in dialog “Create New Board”. The functions and complex devices which are scheduled to be used should already be available in the “Template”, if it is scheduled to delete and create the test program anew. After “CREATE A NEW BOARD” phase, all the changes made with the “Programming Environment” will be inserted only in the program “Function & Macro data base”, and not in the database of the selected Template.

Program data base

These information are related to the test program and are stored in the system database. Here are included all the data used when executing the test program. The data have been created through “Create Pogram” and with the possible changes made by the user or executing in Autodebug or Autolearn modes.

External CAD data files & XML Program data file

This group contains two kinds of information: those related to the board CAD data and those related to the program generated and checked with the information stored when these information were generated with function “Export Data”.

- The information related to the board CAD data are stored into CAD files containing the information required to locate the nodes on the board and the values to be checked. The kind of information is the same as the one[4] included in the “Seica CAD data files” and they can be used to execute the function “Import CAD Data”.
- The information related to the program generated, consist in a description of the program in “*.XML” format. This format can be used when executing “Create Program”, pressing button: “Update Program ...” instead of button: “Create Program !”.

As far as the External CAD data files are concerned, in this manual we refer to CAD Seica format which describes the UUT with the two files: “name.nod” and “name.par” which can be displayed and modified with any editor for text files.

[4] As indicated in “Notes on field MACRO/FUNCTION”, the name of the function associated to the component present in the file “.par”, used in the import data phase, is not exported in the “External CAD data files”. Besides the name of the device associated to the component is different from the name present in the file “.par”, used in the import data phase.

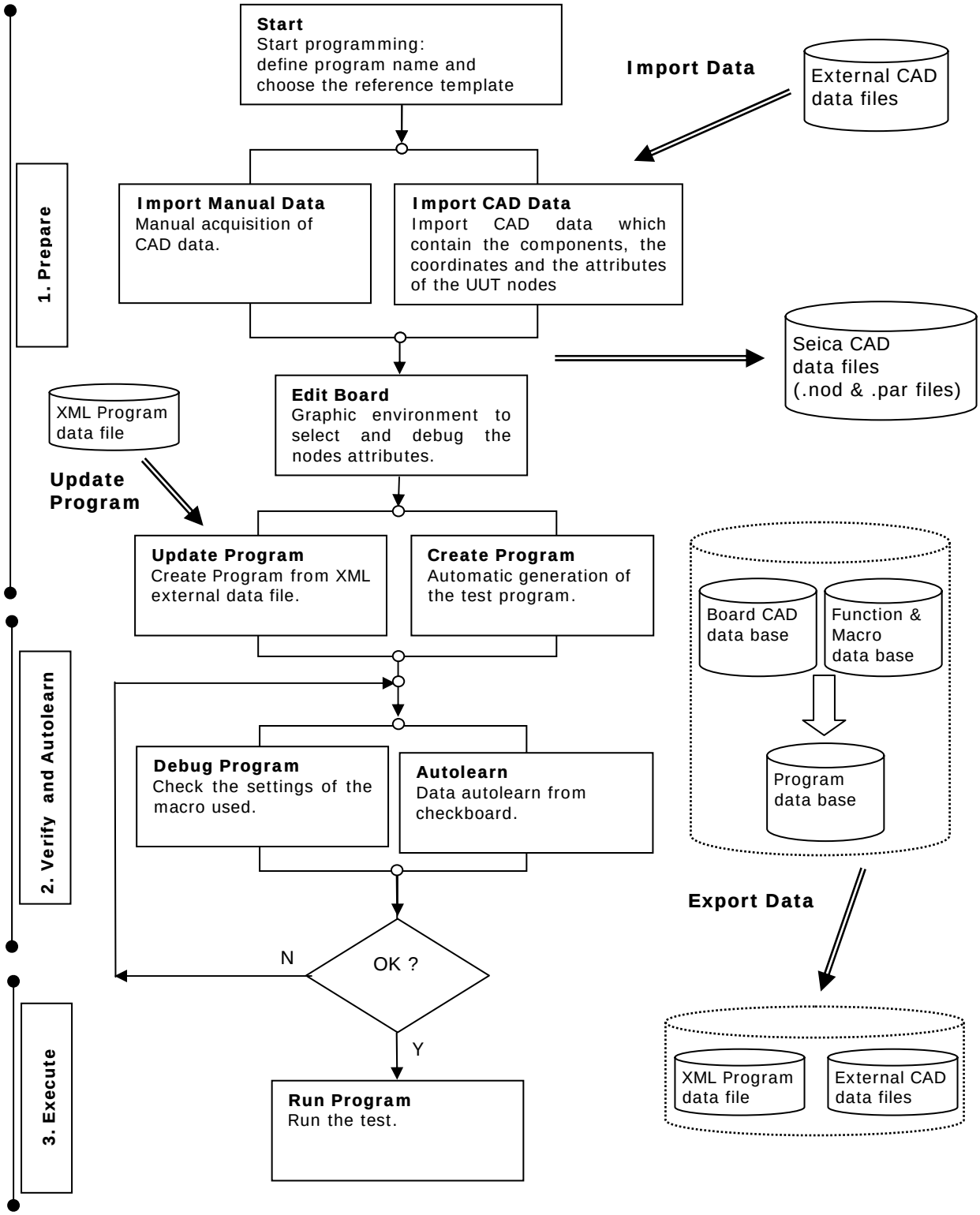


Fig. 1-1 - VIVA: Program Preparation, Verification and Execution

The tables here below show an outline sample record of the “SeicaCAD data files”.

Tab. 1-1 - VIVA: Seica CAD data file (*.nod)

SIGNAL	COMPONENT	PIN	X	Y	PAD	POS	TECN	TEST	CHANNEL
……	……	..	……..	……..	……..	.	.	.	…
“ANODEDI1”	“DI1”	1	170,000	185,000	R39H23	T	T	Y	573
“ANODEDI2”	“DI2”	1	170,000	187,000	R39H23	T	T	Y	574
“CATODEDI1”	“DI1”	2	172,000	185,000	R39H23	T	T	Y	575
“CATODEDI2”	“DI2”	2	172,000	187,000	R39H23	T	T	Y	576
"ACDC1_AC"	"ACDC1"	1	175.000	187.000	R31H23	O	T	Y	652
"ACDC1_DCA"	"ACDC1"	2	175.000	186.000	R31H23	O	T	Y	653
"ACDC1_DCB"	"ACDC1"	3	175.000	185.000	R31H23	O	T	Y	653
"ACDC1_DC+"	"ACDC1"	4	175.000	184.000	R31H23	O	T	Y	654
"ACDC2_AC"	"ACDC2"	1	177.000	187.000	R31H23	O	T	Y	655
"ACDC2_DCA"	"ACDC2"	2	177.000	186.000	R31H23	O	T	Y	656
"ACDC2_DCB"	"ACDC2"	3	177.000	185.000	R31H23	O	T	Y	656
"ACDC2_DC+"	"ACDC2"	4	177.000	184.000	R31H23	O	T	Y	657
……	……	..	……..	……..	……..	.	.	.	…

Tab. 1-2 - VIVA: Seica CAD data file (*.par)

COMPONENT	DEVICE		VALUE	UNIT	TOL+	TOL-	NRPIN		MACRO/Function
……	……		……		……	……	……	……	……
“DI1”	“Diodes”		0.30	V	0.00	0.00	2		
“DI2”	“Diodes”		0.40	V	0.00	0.00	2		
"ACDC1""DiodeResistors"		0.70	V	0.00	0.00	4			
"ACDC2""DiodeResistors"		0.70	V	0.00	0.00	4			
"RL1"	"DRelais"		0.00	V	0.00	0.00	4		
"RL2"	"DRelais"		0.00	V	0.00	0.00	4		
……	……	……		……..	……	……	……		……

According to the records described in the previous tables, the “Import CAD data” phase associates the function “Diode” to devices “Diodes” and“DiodeResistors”.

In the dialog “Assign Function” of tag “Create Program” it is possible to view:

Device	Function	Components
……..	……..	……
DIODERESISTORS	DIODE	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
……..	……..	……

In the Programming Environment, the item “Patterns” of the “Diode” function contains the instruction[5] “DIODE*” which defines the association of function “Diode” to all devices whose name begins with string “DIODE”. (hence also the devices: “Diodes” and “DiodeResistors”).

In the Programming Environment, it is possible to view the structure of “Diode” function:

```
[-] Functions
 +----[-] Diode
        +----[-] Asscociated Macros
        |        +--DIODE
        +----[-] Patterns
                   +--##CR*
                   +--##D*
                   +--##GR*
                   +--##LD*
                   +--##LED*
                   +--DIODE*
```

[5] For further information about the Programming Environment, please refer to the VIVA on-line documentation.

According to the reference CAD data, in the Debug Environment it is possible to view the structure of the “Diode” section.

When the program is executed, the “Diode” section identifies the executon of the function “Diode” on all components belonging to the devices to whom the “Diode” function is associated.

The executon of the “Diode” function on a component identifies the executon of the macros which make up the “Diode” function. (In this case the “Diode” function is made up by a single macro named “DIODE”).

Test sections in the Debug Environment:

```
- DIODE
  +----- - DI1
  |         +--DIODE
  +----- - DI2
  |         +--DIODE
  +----- - ACDC1
  |         +--DIODE
  +----- - ACDC2
            +--DIODE
```

To avoid associating the function “Diode” to the “DIODERESISTORS” devices when creating the program, it is possible to assign the device a name not included in those considered in the directory “Patterns” of the “Diode” function present in the Programming Environment.

For example:

COMPONENT	DEVICE		VALUE	UNIT	TOL+	TOL-	NRPIN	MACRO/Function
…….....	…….		……	….	……..	……..	……	……
“DI1”	“Diodes”		0.30	V	0.00	0.00	2	
“DI2”	“Diodes”		0.40	V	0.00	0.00	2	
"ACDC1"" ***XDiodeResistors*** "		0.70	V	0.00	0.00	4		
"ACDC2"" ***XDiodeResistors*** "		0.70	V	0.00	0.00	4		
"RL1"	"DRelais"		0.00	V	0.00	0.00	4	
"RL2"	"DRelais"		0.00	V	0.00	0.00	4	
……	…….		…….	….	……..	…….	……	……

During the phase “Import Data” the dialog “Assign Funtion” allows to associate a function to the “XDiodeResistors” device. If no association is made, the dialog “Assign Function” in tag “Create Program” shows:

Device	Function	Components
…….	…….	……
XDIODERESISTORS	UNKNOWN	CDC1,ACDC2
ZENERS	ZENER	ZE1,ZE2
…….	……	……

In the Debug Environment, the test sections view shows that ACDC1 and ACDC2 components, belonging to “XDIODERESISTORS” device, are included in section UNKNOWN with no macro associated. So these components will not be checked

```
- UNKNOWN
  +---- ACDC1
  +---- ACDC2
  +----
```

The device “DiodeResistors”, described in Tab. 1-2 and subsequently renamed as “XDiodeResistors”, is an example of a device with no specific test function in the Programming Environment.

The VIVA Environment offers two possible solutions to check this component:

1. Define a new function, made up with the available MACROS which refer to device templates. By associating this function to the device, it is then possible to generate a program to check the components belonging to the device. Since the macros refer to sample devices, in the Debug Environment it is necessary to modify the reference to component pins not corresponding to the actual component. (e.g. for the diode, the macro schedules that pin 1 is associated to the anode and pin 2 to the cathode; the reference pin for resistance are pins 1 and 2, actually, the resistance of the actual component is connected to pins 3 and 4. Hence, after the automatic program generation it is necessary to modify the program to fit the components description included in the CAD data files.
2. Define a complex device made up by a sufficient number of sub-devices which can be checked using the functions included in the “Function & Macro data base”. So, it is possible to generate a test program which does not require subsequent modifications to align the sample component and the real component description.

With solution number 2., the complex device is defined in the “Function & Macro database”. It is then possible to automatically execute all the activities scheduled to create a test program for the real UUT. In the Debug Environment it is possible to execute the verify, autolearn and autodebug without need of alignment between the automatic UUT description and the real UUT description.

The drawings in Fig. 1-2 reproduce the complex components pinout: ACDC1 and ACDC2 and the two possible classifications with a new devide (XDiodeResistors) or with a complex device (ACDC).

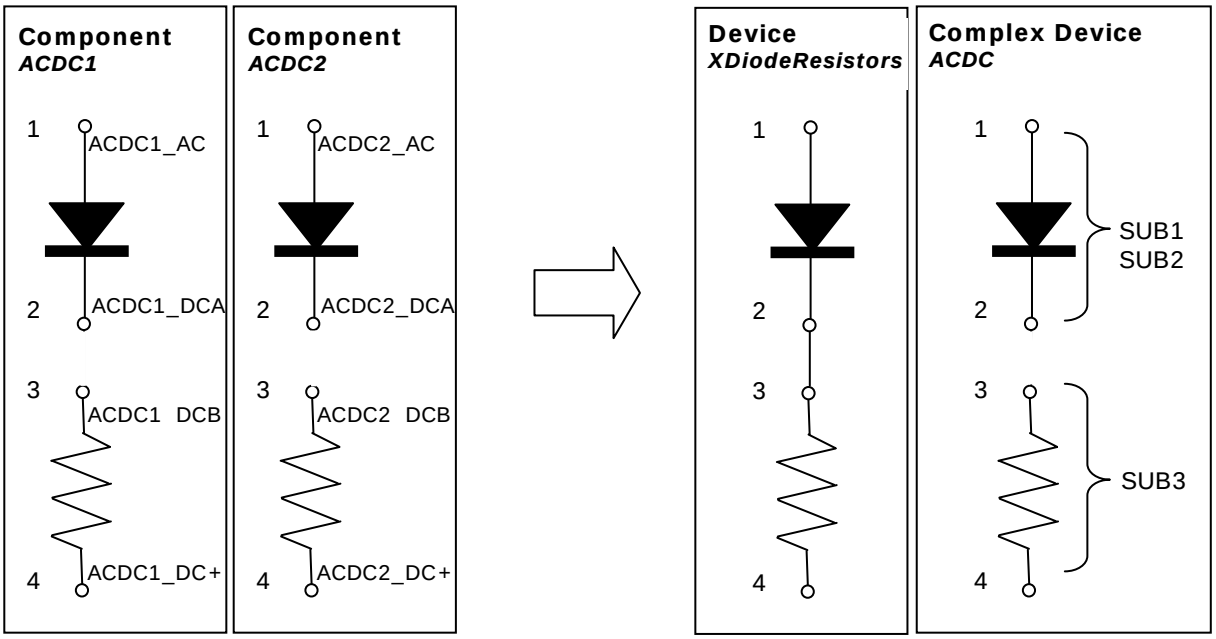


Fig. 1-2 - VIVA: Complex Components, New Device and Complex Device

After the general information provided in this chapter to help and understand the context of complex devices generation, the next chapter describes the activities required in the Programming Environment to generate such devices. For further information, please refer to VIVA SOFTWARE on-line documentation.

1.2 VIVA - Graphical User Interface

To describe dialogs and their parts, we have used the terms shown in Fig. 1-3, which displays the tools available in the graphical user interface, used by VIVA SW to interact with the operator. The system prompts some dialog windows which may contain: toolbars, one or more pages, one or more tools. The pages may contain: toolbars, one or more pages, one or more tools
The dialog displayed in Fig. 1-3 contains:

- A horizontal toolbar at dialog level.
- n pages with several graphic interface tools.

The page shown below contains:

- A vertical toolbar at page level (Toolbox or “Outlook Toolbar).
- A sample of the single controls used by the VIVA software.

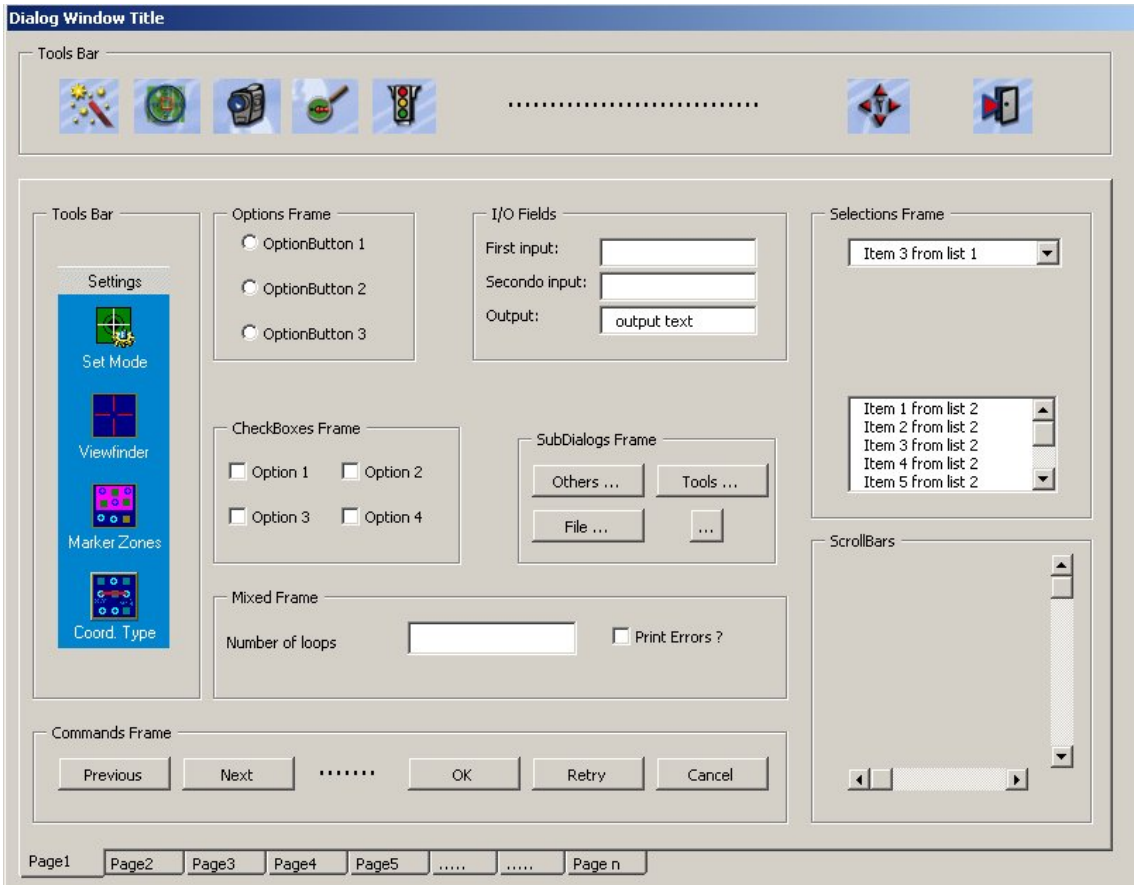


Fig. 1-3 - VIVA: Graphical User Interface

2. Functions and complex devices

The directories or patterns available in the Programming Enviroment are a way to associate functions to components and devices present and described in the UUT CAD data.

Another way to associate functions to component and devices is to use the “Complex Devices”. (In the VIVA GUI, these “Complex Devices”are also called “Complex Components”)[6].

- A complex device identifies a kind of component which may be divided into one or more sub components: SUB1, SUB2, …SUBn.
- Each sub component is defined by:

  1. the number of pins used
  2. for each pin used: by the pin identification number related to the complex device
  3. by the function associated to check the specific sub component. The function must be included in the list of the functions available in the Programming Environment.

- In the VIVA GUI a function named <Complex> is associated to the complex device to distinguish this kind of functions from those available in the Programming Environment.
- In the Debug Environment the name of the section associated to the complex device corresponds to the name of the function associated to the first “sub device” (SUB*1*) of the complex device.

[6] More precisely: a component belongs to a class or kind of components named device. A complex device belongs to a class or kind of components named complex devices.

2.1 Simple devices and complex devices

1. An existing function in the Function & Macro database of the VIVA SW schedules the executon of one or more system macros to check the simple or theoretical devices described in VIVA SW database.
2. The simple devices, existing in “Function & Macro database”, are theoretical devices with a given pin numbering. In other words, the simple devices may be considered as samples which may require a mapping in order to be applied to physical models so that the VIVA SW may interpret them.
3. The physical devices to be checked, may have a pin numbering different from that of the simple device stored in the “Function & Macro database”.
4. With the complex devices it is possible to perform the mapping between the numbering of the simple and the physical device.
5. **It is necessary to assign to every pin of the simple device the number of the corresponding pin on the physical device**. It is important to perform this operation, even though the associated function uses a pin number lower than that defined for the theoretical device.
6. The physical devices may be made up by one or more sub-devices and the mapping between theoretical component and physical component may be defined for each sub-device.

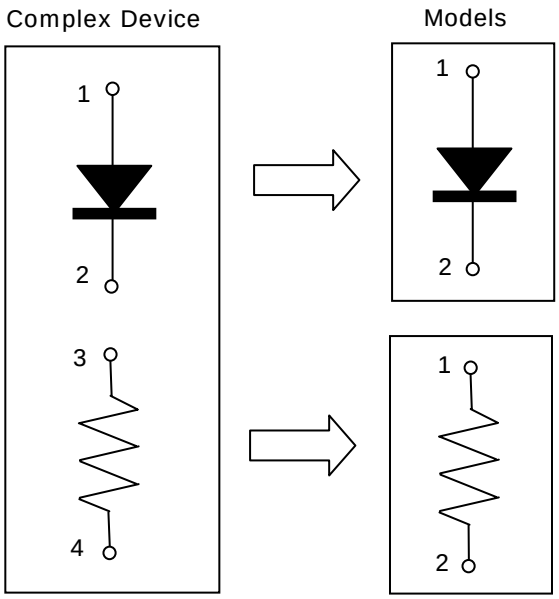
With VIVA SW, a complex device is made up by one or more sub-devices and it is possible to associate a test function to each sub device. The dialog “Complex Component Wizard” allows to define the mapping between simple devices (theoretical devices) and sub-devices (physical devices), besides, it allows to associate the function used to check the sub-device.

The following paragraphs refer two examples of use of complex devices:

- The first one is referred to management of an analog complex device made up by sub-devices for which it is possible to perform the check through functions made up by macros available in the models stored in the Programming Environment.
- The second one is referred to management of a digital complex device where the sub-devices are all the same. Also in this case, it is possible to execute the check through functions made up by macros available in the models stored in Programming Environment.

2.2 Complex device made up by discrete components

The example used is that of the ACDC complex device reproduced in Fig. 1-2 to which the two components ACDC1 and ACDC2 are associated. These two components are described with the two Seica CAD data files reproduced in Tab. 1-1 and Tab. 1-2.



To check the complex device it is necessary to define the function to check the diode sub device, the function to check the resistor sub device (that solves the pin mapping which, for the resistor component is connected to pins 3 and 4 on the complex device while on the sample device it is connected to pins 1 and 2).

The basic steps are:

1. Add the functions required to check all the sub-devices which form the complex device. (FACDC, FACDC_DIODE and FACDC_RESISTOR).

2. Add the complex device which describes all the sub-devices. (DACDC with SUB1, SUB2 and SUB3).

3. Assign a device or a component to a complex device.

This will imply the following changes in the “Board CAD data base”:

1. It is necessary to add to the component list, besides the original component, as many components as many subcomponents (*SUBi*) make up the complex component.

2. To the device list are added:

   - A device to identify the complex device
   - As many devices as many are the functions associated to the subcomponents which make up the complex device.

Below is provided the list of the additional components and devices in the UUT data base after assigning two components (ACDC1 and ACDC2) to a complex device (DACDC) made up by three sub-devices. Each sub-device is checked with the related functions: FACDC, FACDC_DIODE and FACDC_RESISTOR.

It is possible to view the list of components and devices included in “Board CAD data base” through the dialog: “Add Items Wizard”, pressing the button: Add Pins ..., in the page “Board Data” of the Wizard Environment.

<u>Device board list</u>

```
.....
DACDC                ( FACDC )
.....
FACDC                ( FACDC)
FACDC_DIODE          ( FACDC_DIODE )
FACDC_RESISTOR       ( FACDC_RESISTOR )
 .....
```

<u>Component board list</u>

```
.....
ACDC1
ACDC1#1
ACDC1#2
ACDC1#3
ACDC2
ACDC2#1
ACDC2#2
ACDC2#3
.....
```

Fig. 2-1 - VIVA: Devices and Components list for two components of a complex device

2.2.1 Add functions to board data base

This paragraph describes the fundamental steps to insert three functions in the board database.

- Function: “FACDC” Does not contain system macros[7].
- Function: “FACDC_RESISTOR” Containing the “RESISTOR” system macro.
- Function: “FACDC_DIODE” Containing: “DIODE” and “DIODE_INV” system macro.

Programming Environment> Page: “General”

Press button: Add Function

Insert: > “Name:”***FACDC***” ☐ Ignore Macro Selection Criteria Button: OK

Press button: Add Function

Insert: > “Name:”***FACDC_RESISTOR***” ☐ Ignore Macro Selection Criteria Button: OK

Select function: *FACDC_RESISTOR* Select item: “Associated Macros”
In section “Macros:”select macro: *RESISTOR* press button : <<

Select button: Add Function

Insert: > “Name:”***FACDC_DIODE***” ☐ Ignore Macro Selection Criteria Button: OK

Select function: *FACDC_DIODE* Select item: “ Associated Macros”
In section “Macros”: Select macro: *DIODE* Press button: <<
In section “Macros”: select macro: *DIODE_INV* Press button: <<

Press button:” OK > Exit Programming Environment.

When the system prompts the message: “You have modified some code information in Macro Editor.
Do you want that VIVA recreates your Test Program?

Press button:” Yes/Si > to exit Programming Environment and allow the system to create the program.

[7] The FADC function does not execute any macros and might not be inserted. It is used in the e.g. because it improves the readability of the created program. In the Debug Environment there is a section named FADC which identifies the activities executed on that kind of components. Otherwise, the section has the same name of the function used to check the first sub-device of the component (SUB1).

2.2.2 Add a complex device to the board data base

This paragraph describes the fundamental steps to insert in the board database the ACDC complex device made up by three sub-devices:

SUB1 Fictitious sub-device to associate a function to the first sub-device of the complex device. The name of the function is the same as the section displayed in the Debug Environment, identifying the operations performed to check the components which belong to the category identified by the complex device.
SUB2 Identifies the "Diode" sub device of the complex device.
SUB3 Identifies the "Resistor" sub device of the complex device.

Programming Environment> Page: "General" > Section: Component Group Wizards:

Press button: Activate > The dialog: " Complex Components Wizard" is displayed.

Press button: Add Complex Component

In the section "Sub Pin Model" select the template approaching the most the composition of the sub device of the device to be created. (In this case, select the second template from above).

Fill in the following fields in the dialog:

"Name:" ***DACDC***
"Total Pin Number:" ***4***
"Nr. Sub Components:" ***3***
"Function:" ***FACDC***

Button: Add
Button: Close : > exit dialog "Add Complex Component".

Select the *DACDC complex device, then*:

Select: *SUB1* Define the pins used for *SUB1 (pin 1 and pin 2).*
Select the function associated to *SUB1*: *FACDC*
Save the settings related to *SUB1* pressing button: SAVE

Select: *SUB2* Define the pins used for *SUB2 (pin 1 and pin 2)*
Select the function associated to *SUB2*: *FACDC_DIODE*
Set the value and tolerances to be checked on *SUB2*
Save the settings related to *SUB2* pressing button: SAVE

Select: *SUB3* Define the pins used for *SUB3 (pin 3 and pin 4)*
Select the function associated to *SUB3*: *FACDC_RESISTOR*
Set the value and tolerances to be checked on *SUB3*
Save the settings related to sub *SUB3* pressing button: SAVE

Button: Close: > exit dialog "Complex Components Wizard".

Below is displayed the dialog “Complex Component Wizard” related to the three components of the complex device described in the example. Here are reported also the values and tolerances used by functions associated to the components of the complex device.

Complex Components list
.....
[-] DACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

Used pins
1
2
0
0
0
0
0
0

Function
FACDC

☐ Value:	0	H
☐ TOL+:	0	%
☐ TOL-:	0	%

Complex Components list
.....
[-] DACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

Used pins
1
2
0
0
0
0
0
0

Function
FACDC_DIODE

☑ Value:	0.75	V
☑ TOL+:	12	%
☑ TOL-:	12	%

Complex Components list
.....
[-] DACDC
+------Sub1
+------Sub2
+------Sub3
[-] nnnn
.....

Used pins
3
4
0
0
0
0
0
0

Function
FACDC_RESISTOR

☑ Value:	1200	O
☑ TOL+:	5	%
☑ TOL-:	5	%

2.2.3 Assign a device to a complex device

This paragraph describes the fundamental steps to replace an existing device in the board data base, with a complex device DACDC.

Wizard Environment> Page: Create Program

Press button: Assign Function... .

In section "Mode": > Select: ⊙ **Device**
In column "Device": > Select: ***XDIODERESISTORS***

In section: "Assign Complex to Selected Components": **select the complex device** (e.g. ***DACDC***) and associate it to the selected item. (e.g. *XDIODERESISTORS*).

Press button "Apply":> the view switches from configuration: Device Before to configuration: Device After

Button OK, > Exit dialog "Assign Function" and recreate the program!

If an existing device has been associated to a complex device the former device is renamed with the name of the complex device, as shown in the view: Device After.

Device Before:

Device	**Function**	**Components**
.....	.....	.....
XDIODERESISTORS	UNKNOWN	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
.....	.....	.....

Device After:

Device	**Function**	**Components**
.....	.....	.....
DACDC	<Complex>	ACDC1,ACDC2
DIODES	DIODE	DI1,DI2
.....	.....	.....

After program creation, all the macros belonging to the functions of sub components which make up the complex device are associated to the device.

In the Debug Environment it is possible to view the sections[8] where the macros associated to the complex devices are executed.

```
Debug Environment
  ...
- FACDC
  +---- - ACDC1
  |          +-- DIODE : 1-2
  |          +-- DIODE_INV : 1-2
  |          +-- RESISTOR : 3-4
  +---- - ACDC2
             +-- DIODE : 1-2
             +-- DIODE_INV : 1-2
             +-- RESISTOR : 3-4
  ....
```

[8] The name of the section corresponds to that of the function associated to the first sub device of the complex device. In the example described above the "SUB1" sub device has been created together with the empty function "FACDC" to increase the program readability, i.e. to have a section which identifies the complex device and the macros executed on the components belonging to the complex device.

2.2.4 Assign a component to a complex device

This paragraph describes the fundamental steps to assign a component, included in the board database, to a complex device.

Wizard Environment > Page: "Create Program"

Press button: Assign Function... .

In section "Mode": > Select: ⊙ **Component**
In column "Component": > Select: ***ACDC1*** *(or ACDC2)*

In section: "**Assign Complex to Selected Components**": select the complex device (e.g. ***DACDC***) to be associated to the selected item. (e.g. *ACDC1 or ACDC2*).

Press button "Apply": > the view shifts from configuration: Component_Before to configuration: Component_After:

Button OK, > Exit dialog "Assign Function" and recreate the program!

If a component of an existing device has been associated to a complex device, after, the component will belong to the complex device, as shown in the view: Component_After.

Component_Before:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	UNKNOWN	XDIODERESISTORS		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS		0.7		4
.....	.....	.....	.....	.....	...	.....

Component_After:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	**DACDC**		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS		0.7		4
.....	.....	.....	.....	.....	...	.....

After recreating the program, all the macros belonging to the functions of sub components which make up the complex device are associated to the component

In the Debug Environment it is possible to view the sections[9] where the macro associated to complex components are associated.

Debug Environment
(if only ACDC1 has been assigned

```
...
[-] FACDC
  +----[-] ACDC1
  |        +-- DIODE : 1-2
  |        +-- DIODE_INV : 1-2
  |        +-- RESISTOR : 3-4
....
```

Debug Environment
(if both ACDC1 and ACDC2 have been assigned

```
...
[-] FACDC
  +----[-] ACDC1
  |        +-- DIODE : 1-2
  |        +-- DIODE_INV : 1-2
  |        +-- RESISTOR : 3-4
  +----[-] ACDC2
           +-- DIODE : 1-2
           +-- DIODE_INV : 1-2
           +-- RESISTOR : 3-4
....
```

[9] See note in chapter. "Assign a device to a complex device"

2.2.4.1 Assign a function to a component

This paragraph describes the fundamental steps to assign a specific function to check a component included in the board database. This is an **alternative way to check a complex component**. This requires, by the way, after generating the program, to access Debug Environment in order to associate the component pins to be checked to the reference pins existing in the generated program.

Wizard Environment > Page: "Create Program"

Press button : Assign Function... .

In section "Mode": > select: ⊙ **Component**
In column "Component": > Select: ***ACDC2*** *(o ACDC1)*

In section: "**Assign Function to Selected Components**": select the function to be used to check the selected component. (e.g. ***DIODE+RES***)

Press button "Apply": > the view shifts from configuration: Component Before to configuration: Component After.

Button OK, > Exit dialog "Assign Function" and recreate the program!

If a function has been associated to a component, then the component is associated to the new function and the device is renamed <old name>_<component name>, as shown in the view: Component after.

Component Before:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS		0.7		4
.....	.....	.....	.....	.....	...	.....

Component after:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	**DIODE+RES**	**XDIODERESISTORS_ACDC2**		0.7		4
.....	.....	.....	.....	.....	...	.....

If the function "DIODE+RES" has been declared as made up by macros: "DIODE", "DIODE_INV" and "RESISTOR", in the Debug Environment it is possible to view the sections where the macros associated to the complex components are executed.

```
Debug Environment
  ...
- FACDC
  +----- - ACDC1
  |          +--  DIODE : 1-2
  |          +--  DIODE_INV : 1-2
  |          +--  RESISTOR : 3-4
  ....
- DIODE+RES
  +----- - ACDC2
  |          +--  DIODE
  |          +--  DIODE_INV
  |          +--  RESISTOR
  ....
```

2.2.4.2 Assign a macro to a component

This paragraph describes the fundamental steps to assign a macro to check a component included in the board database. This is an **alternative way to check a complex component**. After generating the program, it may be required to access the Debug Environment to associate the pins of the component to be checked to the reference pins included in the created program.

Wizard Environment > Page: "Create Program"

Press button: Assign Function... .

In section "Mode": > Select: ⊙ **Component**
In column "Component": > Select: ***ACDC2*** *(or ACDC1)*

In section: "**Assign Macro to Selected Components**": select the required macro to check the selected component. (e.g. ***NEW_RESISTOR***)

Press button "Apply": > the view shifts from configuration: Component Before to configuration: Component After

Button OK, > Exit dialog "Assign Function" and recreate the program!

If a macro has been associated to a component, then the component is still associated to the previous function, but the macro executed is indicated in field "ForcedMacro", as shown in the view: Component After.

Component Before:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS		0.7		4
.....	.....	.....	.....	.....	...	.....

Component After:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	.....
ACDC1	<Complex>	DACDC		0.7		4
ACDC2	UNKNOWN	XDIODERESISTORS	**NEW_RESISTOR**	0.7		4
.....	.....	.....	.....	.....	...	.....

If the macro "NEW_RESISTOR" has been developed on purpose for the complex component, it may be required to access the Debug Environment to associate the pins of the component to be checked to the reference pins included in the program generated. In Debug Environment it is possible to view the section where the macro associated to complex components is executed.

```
Debug Environment
  ...
- FACDC
  +----- - ACDC1
  |          +-- DIODE : 1-2
  |          +-- DIODE_INV : 1-2
  |          +-- RESISTOR : 3-4
  ....
- UNKNOWN
  +----- - ACDC2
  |          +-- NEW_RESISTOR
  ....
```

2.3 Complex device made up by ic

The operational amplifier has been used as an example of a digital complex component.

The operational amplifier is a high gain DC coupled differential amplifier. It can be used in several applications: non inverting amplifier, inverting amplifier, differential amplifier, buffer, comparator, oscillator, active filter, etc… .
The VIVA SW makes three system macros available to check the main operating conditions of an operational amplifier:

- Comparator (COMPARATOR)
- Operational amplifier (OP_AMP_VF)
- Operational amplifier gain (OP_AMP_GAIN)

The function “OpAmp” available in the VIVA SW Programming Environment schedules the executon of three macros on a sample device made up by an operational amplifier with output on pin 3 and inputs on pin 1 and 2. Normally, in a “in-circuit” context, the operational amplifier may be checked only in one of the three scheduled modes, according to the circuitry connections to the components to be checked. Besides the same ic can contain more instances for the device to be checked and most likely, the real component pins differ from pins 1, 2 and 3 of the sample.

The reference UUT used as a sample schedules four ic, with brand name LF412, with two operational amplifiers each. In the UUT CAD data, the brand name has been assigned to field “Device” and the digital components are identified by the alphanumeric string: “U*ii*”, where “*ii*” is the identification number for the digital components mounted on the UUT.

In the “Programming Environment” of the VIVA SW the digital components used for the example, should be checked with the following features:

U44 belonging to typology or device “LM7812M” can be checked with function “OpAmp” according to directory “Patterns” associated to ”OpAmp” function in the Programming Environment.
U33 belonging to typology or device “LF412” can be checked with function “Digital” according to directory “Patterns” associated to function “Digital” in the Programming Environment.
U34 belonging to typology or device “LF412” can be checked with function “Digital” according to directory “Patterns” associated to function “Digital” in the Programming Environment
U35 belonging to typology or device “LF412” can be checked with function “Digital” according to directory “Patterns” associated to function “Digital” in the Programming Environment
U36 belonging to typology or device “LF412” can be checked with function “Digital” according to directory “Patterns” associated to function “Digital” in the Programming Environment”.

On the ground of the real components and the “in-circuit” context of the components, the features to check the components are the following:

U44 belonging to typology or device “LM7812M” can be checked with function “OpAmp” according to directory “Patterns” associated to function ”OpAmp” in the Programming Environment.
U33 belonging to typology or complex device “DLF412_AMPL” can be checked with the new function “”FOP_AMPL” which schedules the execution of the macro “OP_AMP_GAIN” on the complex device sub devices.
U34 belonging to typology or complex device“DLF412_CMP” can be checked with the new function “FOP_CMP” which schedules the execution of macro “COMPARATOR” on the complex component sub devices.
U35 belonging to typology or complex device “DLF412_VF” can be checked with the new function “FOP_VF” which schedules the execution of macro “OP_AMP_VF” on the complex component sub devices.
U36 belonging to typology or device “DLF412” can be checked with function “OpAmp” which schedules the execution of the three macros: “COMPARATOR”, “OP_AMP_VF” and OP_AMP_GAIN on the complex component sub devices.

Hence, it is necessary to define three functions in order to check the three possible “in_circuit” locations of the components, and a possible empty function in order to improve the program readability:

FOP_AMPL	function to check the operational amplifier used as signal amplifier.
FOP_CMP	function to check the operational amplifier used as signal comparator.
FOP_VF	function to check the operational amplifier used as buffer.
FOP_LF412	empty function to improve the test program readability, grouping in one section all the checks executed on all LF412 kind components.

In order to solve pin mapping problems between real and sample components, it is necessary to define a series of complex devices, which allow to define the pin mapping and the function used to check the sub devices.

DLF412_AMPL	complex device which groups the LF412 components checkable as amplifiers.
DLF412_CMP	complex device which groups the LF412 components checkable as comparators.
DLF412_VF	complex device which groups the LF412 components checkable as buffers.
DLF412	complex device which groups the LF412 checkable as amplifiers, comparators and buffers.

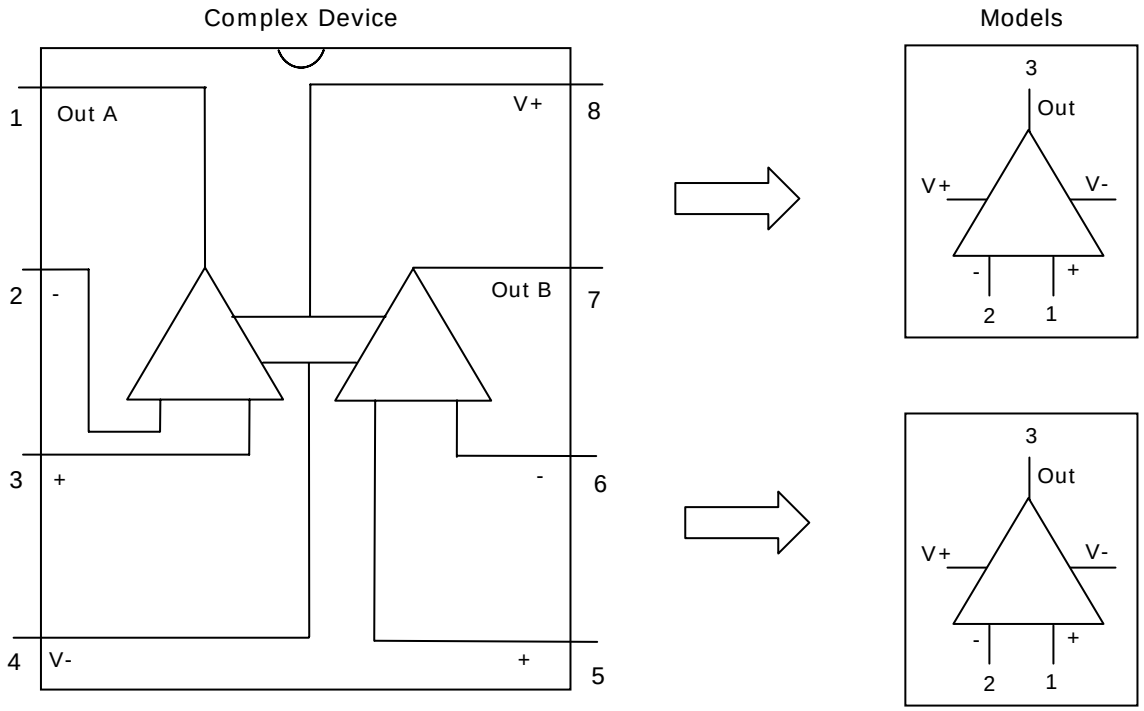


Fig. 2-2 - VIVA: Complex device and VIVA models for Operational Amplifier

To name the new functions and complex devices generated, we have used the prefix “F” and “D” respectively to clearly identify the generated structures.

The drawings reproduced in Fig. 2-3, show a sample of the data displayed in the dialog “Add Item Wizard”[10] which allows to view, from the UUT “Board CAD data base” taken as an example, some reference data used in the following pages of this manual. In brackets you find the function used to check the device operation in the VIVA Environment.

“Add Item Wizard” after “Import Data”:

```
Device board list
.....
LF411          ( Digital )
LF412          ( Digital )
.....
LM7812M        ( OpAmp )*
.....
```

```
Component board list
.....
U33
U34
U35
U36
U37
.....
```

Fig. 2-3 - VIVA: Device and Component list after “import Data”

The functions associated to the device are defined by item “Patterns” included in the description of functions “OpAmp” and “Digital” of the Programming Environment as explained in Fig. 2-4.

Programming Environment functions ”OpAmp” and “Digital””:

```
Function: OpAmp

[-] Functions
 +------[-] OpAmp
        +----[-] Asscociated Macros
        |        +-----COMPARATOR
        |        +-----OP_AMP_VF
        |        +-----OP_AMP_GAIN
        +----[-] Patterns
                 +--------LM*
```

```
Function: Digital

[-] Functions
 +------[-] Digital
        +----[-] Asscociated Macros
        |        +---CLUSTER
        +----[-] Patterns
                 +----##U*
                 +----##IC*
                 +----##74*
                 +----##52*
                 +----##OQ*
                 +----##MC*
                 +----##QFP*
                 +----##DIGITAL*
```

Fig. 2-4 - VIVA: “OpAmp” and “Digital” function from “Funcion & Macro data base”

The Fig. 2-5, reproduces the data available in dialog “Assign Function”, which contains the directories used to create the test program.

The Fig. 2-6, reproduces the view obtained in the Debug Environment of the checks executed on the components indicated in the previous figures.

[10] Selectable with button “Add Pins ...”, of the page: “Board Data” in the Wizard Environment.

“Assign Function – Device” before “Create Program”:

Device	Function	Components
……..	……..	…….
LF411	DIGITAL	U37
LF412	DIGITAL	U33, U34, U35, U36
……..	……..	…….
LM7812M	OPAMP	U44
……..	……..	…….
……..	……..	…….

“Assign Function – Component” before “Create Program”:

Component	Function	Device	ForcedMacro	Value1	…	TotPinNum
……..	……	…….	…….	…….	…	…
U33	DIGITAL	LF412	…….	…….	…	8
U34	DIGITAL	LF412	…….	…….	…	8
U35	DIGITAL	LF412	…….	…….	…	8
U36	DIGITAL	LF412	…….	…….	…	8
U37	DIGITAL	LF411	…….	…….	…	8
……..	……	…….	…….	…….	…	…
U44	OPAMP	LM7812M	…….	…….	…	8
……..	……	…….	…….	…….	…	…

Fig. 2-5 - VIVA: “Assign Function” tables

Debug Environment after “Create Program”:

```
                Debug Environment
   ...
 [-] OPAMP
   +----[-] U44
            +--COMPARATOR
            +--OP_AMP_VF
            +--OP_AMP_GAIN
   ...
 [-] DIGITAL
   +----[-] U37
            +--CLUSTER
  ....
   +----[-] U33
            +--CLUSTER
   +----[-] U34
            +--CLUSTER
   +----[-] U35
            +--CLUSTER
   +----[-] U36
            +--CLUSTER
  ....
```

Fig. 2-6 - VIVA: “Debug Environment”

2.3.1 Add functions to board database

To check the complex components described in the example it is necessary to create four new functions in the Programming Environment. The structure of these functions is indicated in Fig. 2-7

<u>"Programming Environment" after creating the functions for complex devices":</u>

```
Function: FOP_LF412

- Functions
 +------- - FOP_LF412
          +---- - Asscociated Macros
          |
          +---- - Patterns
          ....
```

```
Function: FOP_AMPL

- Functions
 +------- - FOP_AMPL
          +---- - Asscociated Macros
          |     +--------OP_AMP_GAIN
          +---- - Patterns
          ....
```

```
Function: FOP_CMP

- Functions
 +------- - FOP_CMP
          +---- - Asscociated Macros
          |     +--------COMPARATOR
          +---- - Patterns
          ....
```

```
Function: FOP_VF

- Functions
 +------- - FOP_VF
          +---- - Asscociated Macros
          |     +--------OP_AMP_VF
          +---- - Patterns
          ....
```

Fig. 2-7 - VIVA: Functions in Programming Environment

2.3.2 Add a complex device to board database

Fig. 2-8, Fig. 2-9 and Fig. 2-10 reproduce the structure of the three complex devices added to UUT database used as an example:

- DLF412_AMPL
- DLF412_CMP
- DLF412_VF

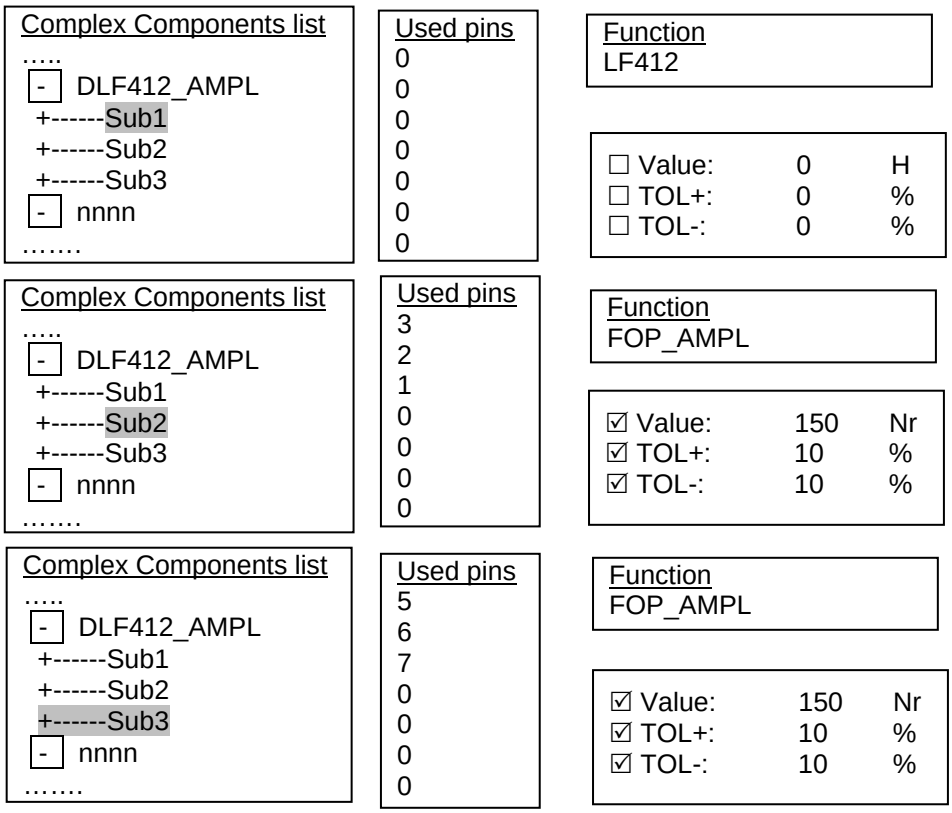


Fig. 2-8 - VIVA: Complex device "Operational Amplifier" used like Amplifier

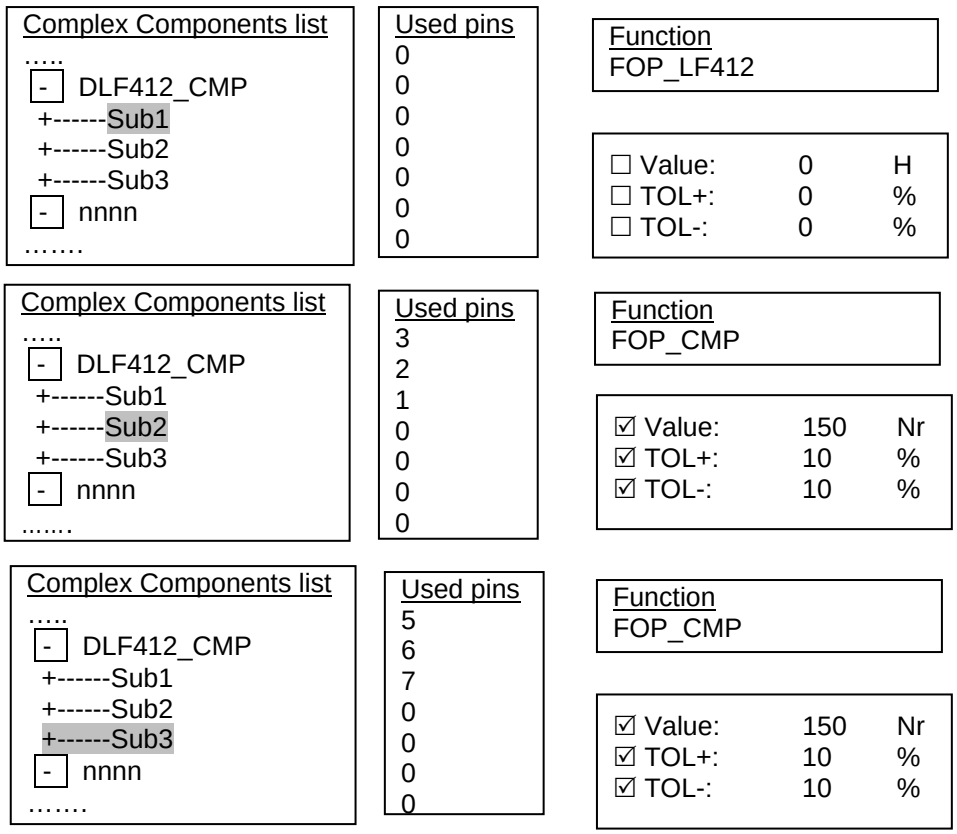


Fig. 2-9 - VIVA: Complex device "Operational Amplifier" used as Comparator

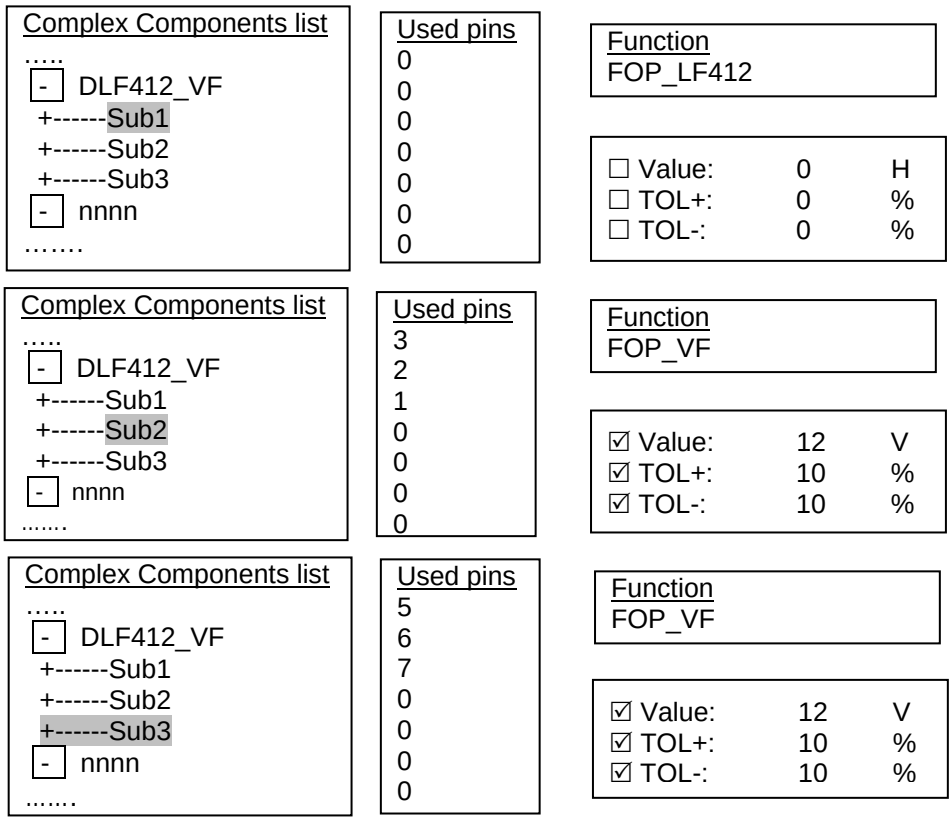


Fig. 2-10 - VIVA: Complex device "Operational Amplifier" used as Voltage Follower

2.3.3 Assign components to complex devices

Fig. 2-11, reproduces the data viewed in dialog “Assign Function”, which contains the directory used to create the test program, after the following associations:

- U33: associated to DLF412_AMPL complex device
- U34: associated to DLF412_CMP complex device
- U35: associated to DLF412_VF complex device

Fig. 2-12, reproduces the view obtained in dialog “Add Item Wizard” which highlights the changes in the UUT components and devices list after associating components to complex devices.

Fig. 2-13, reproduces the view of the checks executed obtained in the Debug Environment, after assigning components to complex devices and recreating the test program.

“Assign Function – Component” after “Assign Function”:

Component	**Function**	**Device**	**ForcedMacro**	**Value1**	**...**	**TotPinNum**
…….	……	…….	…….	…….	…	…
U33	<Complex>	DLF412_AMPL	…….	…….	…	8
U34	<Complex>	DLF412_CMP	…….	…….	…	8
U35	<Complex>	DLF412_VF	…….	…….	…	8
U36	DIGITAL	LF412	…….	…….	…	8
U37	DIGITAL	LF411	…….	…….	…	8
…….	……	…….	…….	…….	…	..
U44	OPAMP	LM7812M	…….	…….	…	8
…….	……	…….	…….	…….	…	..

Fig. 2-11 - VIVA: “Assign Function” Component table

“Add Item Wizard” after “Assign Function”:

Device board list
…..
DLF412_AMPL (FOP_LF412)
DLF412_CMP (FOP_LF412)
DLF412_VF (FOP_LF412)
…..
FOP_AMPL (FOP_AMPL)
FOP_CMP (FOP_CMP)
FOP_LF412 (FOP_LF412)
FOP_VF (FOP_VF)
…..
LF411 (Digital)
LF412 (Digital)
…..
LM7812M (OpAmp)
…..

Component board list
…..
U33
U33#1
U33#2
U33#3
U34
U34#1
U34#2
U34#3
U35
U35#1
U35#2
U35#3
U36
U37
…..

Fig. 2-12 - VIVA: Device and Component list after “Assign Function”

“Debug Environment” after “Create Program”:

```
                    Debug Environment
   ...
 [-] OPAMP
    +----[-] U44
             +--COMPARATOR
             +--OP_AMP_VF
             +--OP_AMP_GAIN
   ...
 [-] DIGITAL
    +----[-] U37
             +--CLUSTER
    +----[-] U33
             +--CLUSTER
   ....

 [+] END

 [-] FOP_LF412
    +----[-] U33
             +--OP_AMP_GAIN : 3-2-1
             +--OP_AMP_GAIN : 5-6-7
    +----[-] U34
             +--COMPARATOR : 3-2-1
             +--COMPARATOR : 5-6-7
    +----[-] U35
             +--OP_AMP_VF : 3-2-1
             +--OP_AMP_VF : 5-6-7
```

Fig. 2-13 - VIVA: “Debug Environment”

2.3.4 Complex device made up by several macros

In order to complete the example described above, this chapter describes the data viewed in the different VIVA environments related to the addition of a complex device (DLF412). The function “OpAmp”, provided with VIVA SW is used to check the sub devices. This function uses the three macros available to check the operational amplifier kind components.

The component associated to complex device “DLF412” is the component U36 and the drawings reproduced below reproduce the new data added to those viewed previously.

“Programming Environment”, function ”OpAmp” used for “DLF412” complex device:

```
Function: OpAmp

[-] Functions
 +------[-] OpAmp
        +----[-] Asscociated Macros
              +--------COMPARATOR
        |     +--------OP_AMP_VF
              +--------OP_AMP_GAIN
```

```
Create a complex device:

  • DLF412
```

Complex device: DLF412

```
Complex Components list
.....
 [-] DLF412
  +------Sub1
  +------Sub2
 [-] nnnn
.......
```

```
Used pins
3
2
1
0
0
0
0
```

```
Function
OpAmp
```

```
☐ Value:    0    H
☐ TOL+:     0    %
☐ TOL-:     0    %
```

```
Complex Components list
.....
 [-] DLF412
  +------Sub1
  +------Sub2
 [-] nnnn
.......
```

```
Used pins
5
6
7
0
0
0
0
```

```
Function
OpAmp
```

```
☐ Value:    150    H
☐ TOL+:     10     %
☐ TOL-:     10     %
```

“Assign Function – Component” after “Assign Function”:

Component	Function	Device	ForcedMacro	Value1	...	TotPinNum
.....	.....	.....	.....	.....	...	...
U33	<Complex>	DLF412_AMPL	.....	.....	...	8
U34	<Complex>	DLF412_CMP	.....	.....	...	8
U35	<Complex>	DLF412_VF	.....	.....	...	8
U36	<Complex>	DLF412	.....	.....	...	8
U37	DIGITAL	LF411	.....	.....	...	8
.....	.....	.....	.....	.....	...	..
U44	OPAMP	LM7812M	.....	.....	...	8
.....	.....	.....	.....	.....	...	..

"Add Item Wizard" after "Assign Function":

```
Device board list
.....
DLF412              ( OpAmp )
DLF412_AMPL         ( FOP_LF412 )
DLF412_CMP          ( FOP_LF412 )
DLF412_VF           ( FOP_LF412 )
.....
FOP_AMPL            ( FOP_AMPL )
FOP_CMP             ( FOP_CMP )
FOP_LF412           ( FOP_LF412 )
FOP_VF              ( FOP_VF )
.....
LF411          ( Digital )
LF412          ( Digital )
.....
LM7812M        ( OpAmp )
.....
```

```
Component board list
.....
U33
U33#1
U33#2
U33#3
U34
U34#1
U34#2
U34#3
U35
U35#1
U35#2
U35#3
U36
U36#1
U36#2
U37
.....
```

"Debug Environment" after "Create Program":

```
                Debug Environment
  ...
  [-] OPAMP
     +----[-] U44
              +--COMPARATOR
              +--OP_AMP_VF
              +--OP_AMP_GAIN
     +----[-] U36
              +--COMPARATOR : 3-2-1
              +--OP_AMP_VF : 3-2-1
              +--OP_AMP_GAIN : 3-2-1
              +--COMPARATOR : 5-6-7
              +--OP_AMP_VF : 5-6-7
              +--OP_AMP_GAIN : 5-6-7
  ...

  [-] DIGITAL
     +----[-] U37
              +--CLUSTER
  ....

  [+] END

  [-] FOP_LF412
     +----[-] U33
              +--OP_AMP_GAIN : 3-2-1
              +--OP_AMP_GAIN : 5-6-7
     +----[-] U34
              +--COMPARATOR : 3-2-1
              +--COMPARATOR : 5-6-7
     +----[-] U35
              +--OP_AMP_VF : 3-2-1
              +--OP_AMP_VF : 5-6-7
```

3. Bibliography

[1] VIVA Macro di sistema
Code: MA-VIVAMSIT-00 (05-24-2007) Seica S.p.A.

VIVA System macro
Code: MA-VI- VIVAMSEN-01 (05-08-2007) Seica S.p.A.

[2] Modulo ACL
Code: MA-ACLI00IT-05 (06-21-2007) Seica S.p.A.

ACL Module
Code: MA-ACLI00EN-03 (04-24-2007) Seica S.p.A.

[3] VIVA Language MA-VIVAVLIT-00
Code: MA-VIVAVLIT-00 (08-03-2006) Seica S.p.A..

VIVA Language MA-VIVAVLEN-00
Code: MA-VIVAVLIT-00 (01-31-2006) Seica S.p.A..